Reviewer Pack — Minimal Number of Automorphism Groups and Circuit Entropy Va…

Entry e80d60e179ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 03:28:43 UTC

Minimal Number of Automorphism Groups and Circuit Entropy Variance

Entry ID: `e80d60e179ec` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 03:28:43 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Automorphism Groups) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> The variance of the number of automorphism groups for a given boolean circuit is upper-bounded by Θ(n^2), where n is the size of the circuit.

Rationale (proposer's reasoning):

> Automorphism groups can capture symmetries in circuits, and their minimal count could potentially reveal intrinsic complexity. A connection between this invariant and circuit entropy variance could provide insights into the structure of computational tasks.

Taxonomy category: `AUTOMORPHISM_GROUP_NUMBER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ce343762c144d4c6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the empirical mean of the entropy variance of automorphism group counts for circuits with n inputs is ≤ Θ(n^2) across all 30 seeds, and the standard deviation is ≤ 0.1 * Θ(n^2). It is falsified if either the empirical mean exceeds Θ(n^2) by more than 0.2 * Θ(n^2) or the standard deviation exceeds 0.2 * Θ(n^2).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"automorphism group variance" AND "boolean circuit complexity"` - `"circuit entropy variance" IN group_theory` - `"Boolean circuit" AND automorphism_groups`

Top relevant hits considered: - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/2509.13291v1] Towards an Embodied Composition Framework for Organizing Immersive Computational Notebooks - [s2:1005.2254] On Universal Complexity Measures - [s2:02e50f5a2eedb0265d28b4bfae4f975197f6cf84] A Study of Width Bounded Arithmetic Circuits and the Complexity of Matroid Isomorphism[HBNI TH 17]

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def calculate_entropy_variance(counts):
        n = len(counts)
        if n == 0:
            return 0.0
        mean = sum(counts) / n
        variance = sum((x - mean) ** 2 for x in counts) / n
        return variance

    def generate_boolean_circuit(n):
        circuit = []
        for _ in range(2**n):
            circuit.append(random.choice([0, 1]))
        return circuit

    def calculate_automorphism_group_count(circuit):
        # Placeholder function to simulate automorphism group count calculation
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, n)

    all_counts = []
    for _ in range(30):  # Sample 30 instances per seed
        circuit = generate_boolean_circuit(n)
```

```
        automorphism_count = calculate_automorphism_group_count(circuit)
        all_counts.append(automorphism_count)

    variance = calculate_entropy_variance(all_counts)
    mean = sum(all_counts) / len(all_counts)

    conjecture_holds = mean <= n**2 and variance <= 0.1 * n**2
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Entropy Variance of Automorphism Group Counts",
        "metric_value": variance,
        "instances_tested": len(all_counts),
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_variance = sum(result["metric_value"] for result in results) / len(results)
    std_variance = math.sqrt(sum((result["metric_value"] - mean_variance) ** 2 for result in resul
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_variance} std={std_variance} support_fraction={suppor
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34088764.py", line 67, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34088764.py", line 42, in run_trial
    circuit = generate_boolean_circuit(n)
                                       ^
NameError: name 'n' is not defined
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture's empirical mean and standard deviation meet the support conditions. | next: Debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12654
2	preregistration	ollama_remote	glm4:latest	0	0	9905
3	novelty	ollama_remote	glm4:latest	0	0	8381
4	novelty	ollama_remote	glm4:latest	0	0	9616
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11935
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8929
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9737
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7935
9	judge	ollama_remote	glm4:latest	0	0	16074

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 95168 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e80d60e179ec.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e80d60e179ec.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e80d60e179ec.tar.gz` (if generated)